produces:

```
Comparing self == value
true
Comparing self != value
false
```

You can use a Ruby range as a boolean expression. A range such as `exp1..exp2` will evaluate as false until `exp1` becomes true. The range will then evaluate as true until `exp2` becomes true. Once this happens, the range resets, ready to fire again.

if and unless Expressions

An `if` expression in Ruby is pretty similar to `if` statements in other languages:

```
if artist == "Gillespie" then
  handle = "Dizzy"
elsif artist == "Parker" then
  handle = "Bird"
else
  handle = "unknown"
end
```

The `then` keyword is optional if you lay out your statements on multiple lines:

```
if artist == "Gillespie"
  handle = "Dizzy"
elsif artist == "Parker"
  handle = "Bird"
else
  handle = "unknown"
end
```

However, if you want to lay out your code more tightly, you must separate the boolean expression from the following statements with the `then` keyword:

```
if artist == "Gillespie" then handle = "Dizzy"
elsif artist == "Parker" then handle = "Bird"
else handle = "unknown"
end
```

You can have zero or more `elsif` clauses and an optional `else` clause. And notice that there's no `e` in the middle of `elsif`.

As we've said before, an `if` statement is an expression—it returns a value. You don't have to use the value of an `if` statement, but it can come in handy:

```
handle = if artist == "Gillespie"
           "Dizzy"
         elsif artist == "Parker"
           "Bird"
         else
           "unknown"
         end
```

Ruby also has a negated form of the `if` statement:

```
unless volume.nil?
```